

Author

Name : Aadil Ahmed

Roll number : 23f2004454

student email : 23f2004454@ds.study.iitm.ac.in

About me : I am Aadil Ahmed with a passion to combine tech , data and business to enhance the society . This is one step in that direction.

AI/LLM usage

20 - 30 % for suggestion and learning the intricacy of the code

Project Details

Project Title: Placement Portal Application

Problem Statement: Currently, many institutes rely on spreadsheets, emails, or manual processes, which makes it difficult to manage company approvals, track student applications, avoid duplicate registrations, and maintain placement records.

You are required to build a **Placement Portal Application** web application that allows **Admin (Institute), Company, and students** to interact with the system based on their roles.

Approach:

The app was built using Flask as the backend framework with a modular structure. It has role based login system for efficient login of company ,admin and student

Technologies used

Flask for application back-end

Jinja2 templating, **HTML**, **CSS** and Bootstrap for application front-end

SQLite for databas

DB Schema Design

https://drive.google.com/file/d/1Fwpvi9a1fWpmngvuN_auUBBV4uvm1TEI/view?usp=sharing

The above link contains the whole ER diagram for the placement portal project

Got it. Now I'll rewrite sections 6 and 7 exactly in that format but for **our Placement Portal project**.

Architecture and Features

Architecture Overview:

- `app.py` — Main Flask application entry point. Initialises app, registers blueprints, connects database, seeds default admin on first run.
- `extensions.py` — Holds shared `db` (SQLAlchemy) and `login_manager` (Flask-Login) instances. Kept separate to avoid circular imports.
- `models.py` — All 5 database models: Admin, Company, Student, PlacementDrive, Application.
- `routes/` — Flask Blueprints split by role:
 - `auth.py` — Login, Register, Logout
 - `admin.py` — All admin actions
 - `company.py` — All company actions
 - `student.py` — All student actions
- `templates/` — Jinja2 HTML templates organised by role (`auth/`, `admin/`, `company/`, `student/`). All extend a shared `base.html` layout.

Implemented Features:

- Role-based login for Admin, Company, and Student with session handling via Flask-Login
- Company and Student registration with secure password hashing using Werkzeug
- Admin approval workflow for company registrations and placement drives
- Admin ability to search, edit, blacklist, and delete students and companies

Company dashboard with drive creation, editing, closing, and deletion

Video link :  finalrec.mp4

API Resource Endpoints

Endpoint	Method	Description
<code>/login</code>	POST	Authenticate user (Admin/Company/Student) and create session
<code>/register</code>	POST	Register a new Company or Student account
<code>/logout</code>	GET	Clear session and log out current user

<code>/admin/dashboard</code>	GET	Fetch stats — total students, companies, drives, applications
<code>/admin/company/<id>/approve</code>	GET	Approve a pending company registration

•